МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Вычислительная математика»

Тема: Метод простых итераций

Студент гр. 4342 _____ Тиссен М.Ю.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить метод простых итераций для численного решения уравнения f(x)=0, получить приближённое значение корня с заданной точностью, а также исследовать скорость сходимости и обусловленность метода.

Задание.

Метод простых итераций решения уравнения f(x) = 0 состоит в замене исходного урав нения эквивалентным ему уравнением x = φ(x) и построении последовательности $x_{n+1} = \varphi(x_n)$, сходящейся при n->∞ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1}, вычисляется y = $\varphi(x_{n-1})$. Если |y - x_{n-1}| > ε, то полагается x_n = y и выполняется очередная итерация. Если же |y - x_{n-1}| < ε, то вычисления заканчиваются и за приближенное значение корня принимается величина x_n= y. Погрешность результата вычислений зависит от знака производной $\varphi'(x)$: при $\varphi'(x) > 0$ погрешность определения корня составляет qε/1-q, а при $\varphi'(x) < 0$ погрешность не превышает ε. Здесь q- число, такое, что | $\varphi'(x)$ | ≤ q < 1 на отрезке [a,b]. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции φ(x) в уравнении x = φ(x), эквивалентном исходному. Функцию φ(x) необходимо подбирать так, чтобы | $\varphi'(x)$ | ≤ q < 1. Это обусловливается тем, что если φ'(x)>0 на отрезке [a,b], то последовательные приближения xn+1=φ(xn) будут колебаться около корня c, если же φ'(x)<0, то последовательные приближения будут сходиться к корню c монотонно. Следует также помнить, что скорость сходимости последовательности {xn} к корню c функции f(x) тем выше, чем меньше число q.

В лабораторной работе № 6 предлагается, используя программы функции ITER и Round из файла methods.cpp (файл заголовков methods.h,

директория LIBRI1), найти корень уравнения f(x) = 0 с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции f(x) задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №6 должен быть следующим:

1) Графически или аналитически отделить корень уравнения f(x) = 0.

2) Преобразовать уравнение f(x) = 0 к виду x = φ(x) так, чтобы в некоторой окрестно-
сти [Left, Right] корень производная φ'(x) удовлетворяла условию |φ'(x)|≤1. При этом следует иметь в виду, что чем меньше величина q, тем быстрее последовательные приближения сходятся к корню.

3) Выбрать начальное приближение, лежащее на [Left, Right].

4) Составить подпрограмму для вычисления значений φ(x), φ'(x), предусмотреть округление вычисленных значений с точностью Delta.

5) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к программам φ(x), φ'(x) и ITER в индикацию результатов.

6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Текст программы - функции ITER, позволяющей вычислить корень уравнения $x=\varphi(x)$ для любой функции, которая удовлетворяет достаточным условиям сходимости метода, приводится ниже.

Выполнение работы.

1. Отделение (изоляция) корня уравнения.

Для функции $f(x) = e^{(1/x^2)} - \ln(x)$ рассматривается область $x > 0$. На этой области функция непрерывна и дифференцируема.

Производная исходной функции имеет вид

$$f'(x) = -2e^{(1/x^2)}/x^3 - 1/x.$$

При $x > 0$ оба слагаемых отрицательны, следовательно, $f'(x) < 0$ и функция строго убывает на всей области определения. Это означает, что на любом отрезке, где обнаружена смена знака, корень будет единственным.

Проверим значения функции на концах отрезка [3; 4]:

$$f(3) = e^{(1/9)} - \ln(3) \approx 0{,}018906780074 > 0,$$

$$f(4) = e^{(1/16)} - \ln(4) \approx -0{,}321799902202 < 0.$$

Так как $f(3)\cdot f(4) < 0$, то по теореме Больцано на интервале (3; 4) существует корень уравнения $f(x) = 0$. Из строгой монотонности функции следует, что этот корень единственный. Следовательно, в качестве рабочего отрезка изоляции можно взять [3; 4].

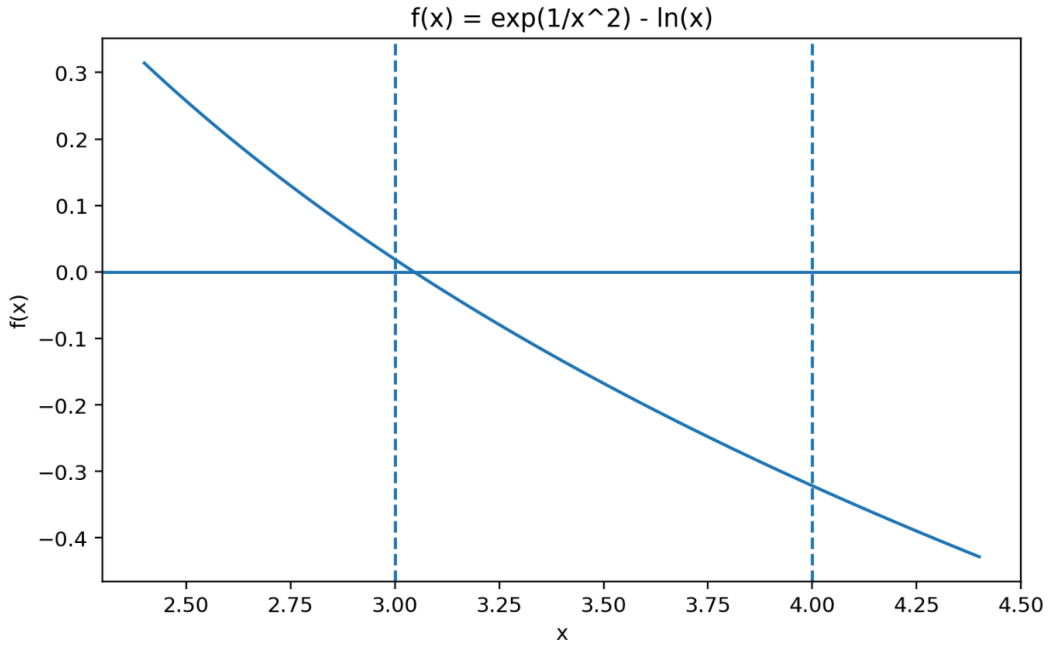

Рисунок 1 — График функции $f(x) = e^{(1/x^2)} - \ln(x)$ и отрезок изоляции корня [3; 4].

2. Преобразование уравнения к виду $x = \varphi(x)$.

Для применения метода простых итераций исходное уравнение перепишем в форме

$$x = \varphi(x), \quad \varphi(x) = e^{e^{1/x^2}}.$$

Действительно, из равенства $e^{1/x^2} = \ln(x)$ получаем $x = e^{e^{1/x^2}}$.

Для обеспечения сходимости метода выберем окрестность корня [3; 3,1]. На этом отрезке функция $\varphi(x)$ непрерывна и принимает значения

$$\varphi(3) \approx 3{,}057259935022, \quad \varphi(3{,}1) \approx 3{,}033342406910.$$

Следовательно, $\varphi([3;\ 3{,}1]) \subset [3;\ 3{,}1]$. Производная итерационной функции равна

$$\varphi'(x) = -2 \cdot e^{e^{1/x^2}} \cdot e^{1/x^2} / x^3.$$

На отрезке [3; 3,1] выполняется оценка $|\varphi'(x)| \le q \approx 0{,}253078 < 1$. Следовательно, достаточное условие сходимости метода простых итераций выполнено.

Так как производная $\varphi'(x)$ на выбранном интервале отрицательна, последовательные приближения сходятся к корню с небольшими колебаниями.

В качестве рабочего интервала итераций принимаем [3; 3,1].

В качестве начального приближения удобно взять середину этого отрезка.

$x_0 = (3 + 3{,}1) / 2 = 3{,}05$.

3. Подпрограммы вычисления $f(x)$, $\varphi(x)$ и $\varphi'(x)$.

В главной программе реализованы три функции: исходная функция $f(x) = e^{1/x^2} - \ln(x)$, её производная $f'(x)$ и итерационная функция $\varphi(x) = e^{e^{1/x^2}}$. Значение $\varphi'(x)$ вычисляется по формуле $\varphi'(x) = -2 \cdot e^{e^{1/x^2}} \cdot e^{1/x^2} / x^3$.

Так как функция содержит логарифм, в программе дополнительно контролируется область определения: при $x \le 0$ вычисления прерываются сообщением об ошибке.

Для исследования чувствительности метода к ошибкам вычислений используется подпрограмма Round(value, Δ), округляющая значение до ближайшего числа, кратного Δ. При Δ = 0 округление не выполняется.

В экспериментах по обусловленности округление применяется к значению итерационной функции φ(x). Это позволяет проследить, как конечная точность промежуточных вычислений влияет на число шагов и на итоговую ошибку найденного корня.

4. Подпрограммы Round, EstimateQ и ITER.

В файле methods.cpp реализованы три основные процедуры. Функция Round выполняет округление значения с шагом Δ. Функция EstimateQ оценивает величину $q = \max|\varphi'(x)|$ на отрезке [Left; Right] по равномерной сетке. Основная процедура ITER строит последовательность приближений

$$x_{\square+1} = \varphi(x_{\square}).$$

На каждом шаге вычисляется новое значение $x_{\square+1}$ и разность $|x_{\square+1} - x_{\square}|$. Если $|x_{\square+1} - x_{\square}| < \mathrm{Eps}$, итерационный процесс прекращается, а значение $x_{\square+1}$ принимается за приближённый корень. Если условие не выполнено, выполняется следующий шаг.

Для рассматриваемой функции метод имеет линейную скорость сходимости, поскольку $q < 1$ на всём рабочем интервале. Практически это означает устойчивое уменьшение ошибки и сравнительно умеренное число итераций.

5. Головная программа и организация экспериментов.

Главная программа задаёт параметры задачи: интервал итераций [3; 3,1], начальное приближение $x_0 = 3{,}05$ и предельное число итераций Nmax

= 1000. Сначала выполняется опорный расчёт без округления при Eps = 10^{-14}, который даёт эталонное значение корня xref.

Далее проводятся два набора вычислительных экспериментов. Первый набор предназначен для исследования скорости сходимости: точность Eps меняется от 10^{-1} до 10^{-8}, после чего для каждого запуска сохраняются число итераций, найденный корень и абсолютная ошибка $|x - xref|$.

Второй набор посвящён обусловленности метода. При фиксированной точности Eps = 10^{-8} рассматриваются различные значения шага округления Δ. Для каждого Δ сохраняются число итераций, полученное значение Δx, абсолютная ошибка $|\Delta x - xref|$ и невязка $|f(\Delta x)|$. Результаты экспериментов записываются в CSV-файлы, а затем по ним строятся графики в программе plot.py.

6. Результаты вычислительных экспериментов.

Опорный расчёт без округления дал значение

$$xref \approx 3.045936104786176.$$

Для получения этого значения методу простых итераций потребовалось 20 шагов при Eps = 10^{-14}. Ниже приведены основные результаты вычислительного эксперимента.

6.1. Исследование скорости сходимости.

В таблице 1 показано, как меняется число итераций при уменьшении требуемой точности Eps. Видно, что метод сходится устойчиво: при ужесточении требования по точности количество шагов возрастает постепенно, что соответствует линейному характеру сходимости.

Таблица 1 — Результаты исследования скорости сходимости метода простых итераций.

Eps	N	x	|x − xref|	|f(x)|
0.1	1	3.04496	9.74e-4	3.96e-4
0.01	1	3.04496	9.74e-4	3.96e-4
0.001	3	3.04588	5.61e-5	2.29e-5
0.0001	4	3.04595	1.35e-5	5.49e-6
1e-05	6	3.04594	7.77e-7	3.16e-7
1e-06	7	3.04594	1.87e-7	7.60e-8
1e-07	9	3.04594	1.08e-8	4.38e-9
1e-08	11	3.04594	6.20e-10	2.52e-10

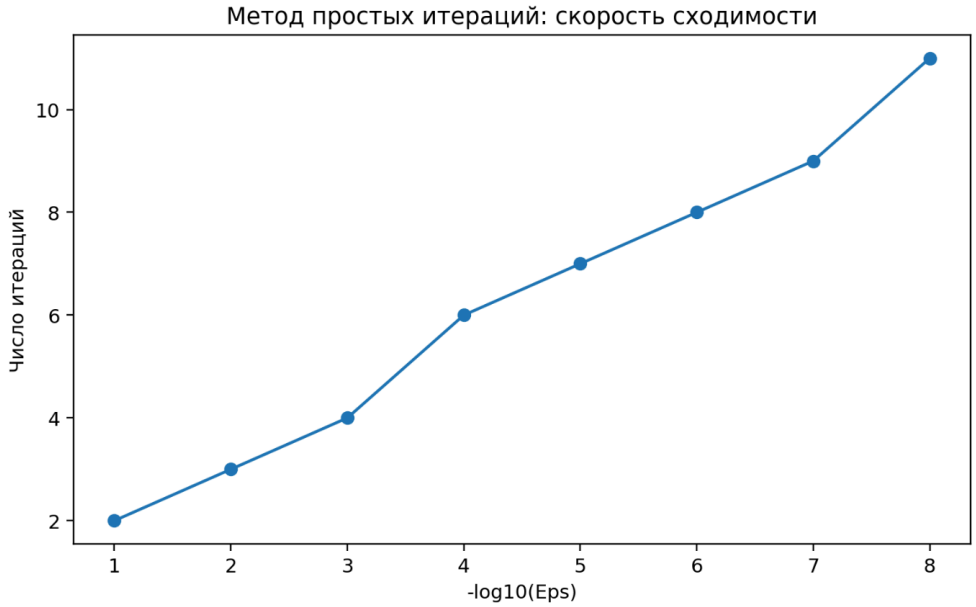

Рисунок 2 — Зависимость числа итераций метода простых итераций от требуемой точности Eps.

По графику видно, что при уменьшении Eps число итераций возрастает почти линейно по отношению к величине $-\log_{10}(\mathrm{Eps})$. Это типичный признак линейной сходимости: на каждом шаге ошибка уменьшается примерно в постоянное число раз, которое определяется величиной q.

6.2. Исследование чувствительности к округлению.

В таблице 2 приведены результаты при фиксированной точности Eps = 10^{-8} и различных значениях Δ. Абсолютная ошибка вычислялась по

формуле |Δx − xref|. Эксперимент показывает, что грубое округление существенно ухудшает результат, а при достаточно малых Δ метод снова работает устойчиво.

Таблица 2 — Влияние округления значений итерационной функции на результат метода.

Δ	**Статус**	**N**	**Δx**	**|Δx − xref|**	**|f(Δx)|**
0.1	nmax	1000	3.1	5.41e-2	2.17e-2
0.01	nmax	1000	3.05	4.06e-3	1.65e-3
0.001	ok	3	3.046	6.39e-5	2.60e-5
0.0001	ok	4	3.0459	3.61e-5	1.47e-5
1e-05	ok	7	3.04594	3.90e-6	1.59e-6
1e-06	ok	8	3.045936	1.05e-7	4.27e-8
1e-07	ok	10	3.045936	4.79e-9	1.95e-9
1e-08	ok	11	3.045936	4.79e-9	1.95e-9

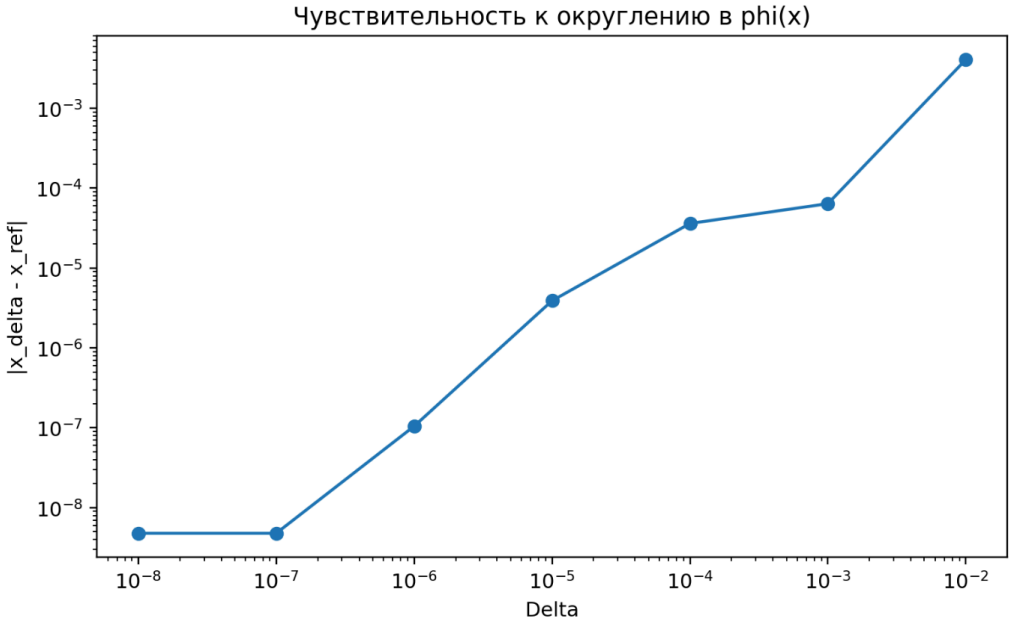

Рисунок 3 — Зависимость абсолютной ошибки |Δx − xref| от параметра Δ.

С уменьшением Δ абсолютная ошибка в целом снижается. Уже при Δ ≤ 10^{-5} найденное значение корня становится очень близким к эталонному, а

при $\Delta = 10^{-7}$ и $\Delta = 10^{-8}$ отличие составляет всего несколько единиц последнего знака.

Следует отметить, что при $\Delta = 10^{-1}$ метод не сходится к корню: возникает двухточечный цикл 3,0 ↔ 3,1, поэтому за предельное число шагов Nmax условие остановки не выполняется. Этот результат хорошо демонстрирует чувствительность метода простых итераций к грубому округлению промежуточных вычислений.

Вывод.

В ходе лабораторной работы для функции $f(x) = e^{(1/x^2)} - \ln(x)$ был реализован метод простых итераций. Корень уравнения был изолирован на интервале [3; 4], после чего исходное уравнение было преобразовано к виду $x = \varphi(x)$, где $\varphi(x) = e^{(e^{(1/x^2)})}$.

Для выбранной итерационной функции на интервале [3; 3,1] выполнено условие сходимости $|\varphi'(x)| \leq q \approx 0{,}253078 < 1$, поэтому метод сходится. Эталонное значение корня составило $x_{ref} \approx 3{,}045936104786176$.

Численные эксперименты показали, что метод обладает линейной скоростью сходимости: при увеличении требуемой точности число итераций растёт постепенно, а при $Eps = 10^{-6}$ требуется 7 шагов.

Исследование обусловленности показало, что при малых значениях Δ вычисленный корень практически совпадает с эталонным, тогда как грубое округление заметно искажает результат и может даже нарушить сходимость метода. Следовательно, для данной функции метод простых итераций применим и даёт устойчивые результаты при корректном выборе $\varphi(x)$ и достаточной точности вычислений.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#pragma once

#include <functional>

struct IterResult {
    double x;         // последнее приближение
    double lastDiff;  // |x_(n+1) - x_n| на последнем шаге
    int iters;        // число выполненных итераций
    bool ok;          // true, если достигнут критерий остановки
};

// Округление значения до ближайшего числа, кратного delta.
// Если delta <= 0, значение возвращается без изменений.
double Round(double value, double delta);

// Оценка q = max |phi'(x)| на [left, right] равномерным перебором точек.
double EstimateQ(const std::function<double(double)>& dphi,
                 double left,
                 double right,
                 int samples = 10000);

// Метод простых итераций для уравнения x = phi(x).
// phi   - итерационная функция
// x0    - начальное приближение
// eps   - требуемая точность по условию |x_(n+1) - x_n|
// nmax  - максимально допустимое число итераций
IterResult ITER(const std::function<double(double)>& phi,
                double x0,
                double eps,
                int nmax = 1000);
```

Название файла: methods.cpp

```
#include "methods.h"

#include <cmath>

// ==============================================================
// Округление значения до ближайшего числа, кратного delta
// ==============================================================
```

```
double Round(double value, double delta) {
    if (delta <= 0.0) {
        return value;
    }
    return std::round(value / delta) * delta;
}

// ==============================================================
// Оценка q = max |phi'(x)| на [left, right]
// ==============================================================
double EstimateQ(const std::function<double(double)>& dphi,
                 double left,
                 double right,
                 int samples) {
    if (samples < 1) {
        samples = 1;
    }

    double q = 0.0;
    for (int i = 0; i <= samples; ++i) {
        const double t = static_cast<double>(i) / samples;
        const double x = left + (right - left) * t;
        const double value = std::fabs(dphi(x));
        if (value > q) {
            q = value;
        }
    }
    return q;
}

// ==============================================================
// Метод простых итераций
// ==============================================================
IterResult ITER(const std::function<double(double)>& phi,
                double x0,
                double eps,
                int nmax) {
    IterResult result{};
    result.x = x0;
    result.lastDiff = 0.0;
    result.iters = 0;
    result.ok = false;
```

```
    double x = x0;
    for (int n = 1; n <= nmax; ++n) {
        const double y = phi(x);
        const double diff = std::fabs(y - x);

        result.x = y;
        result.lastDiff = diff;
        result.iters = n;

        if (diff < eps) {
            result.ok = true;
            return result;
        }

        x = y;
    }

    return result;
}
```

Название файла: main.cpp

```
#include "methods.h"

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr double LEFT = 3.0;
constexpr double RIGHT = 4.0;
constexpr double X0 = 3.5;
constexpr int NMAX = 1000;

// Для f(x) = exp(1 / x^2) - ln(x) на [3, 4]
// m = min |f'(x)|, M = max |f'(x)|, lambda = 2 / (m + M)
constexpr double M = 0.41611252361050843;
```

```
constexpr double m = 0.28326545184118310;
constexpr double LAMBDA = 2.0 / (m + M);

struct SpeedRow {
    double eps;
    int iters;
    double x;
    double absError;
    double residual;
};

struct ConditioningRow {
    double delta;
    int iters;
    double x;
    double absError;
    double residual;
    bool ok;
};

double f(double x) {
    if (x <= 0.0) {
        throw std::domain_error("f(x) is undefined for x <= 0");
    }
    return std::exp(1.0 / (x * x)) - std::log(x);
}

double fp(double x) {
    if (x <= 0.0) {
        throw std::domain_error("f'(x) is undefined for x <= 0");
    }
    return -2.0 * std::exp(1.0 / (x * x)) / (x * x * x) - 1.0 / x;
}

double phi(double x) {
    return x + LAMBDA * f(x);
}

double dphi(double x) {
    return 1.0 + LAMBDA * fp(x);
}
```

```
double phiRounded(double x, double delta) {
    return Round(phi(x), delta);
}

void WriteSpeedCsv(const std::string& filename,
                   const std::vector<SpeedRow>& rows) {
    std::ofstream out(filename);
    out << "eps,iters,x,abs_error,residual\n";
    out << std::setprecision(17);
    for (const auto& row : rows) {
        out << row.eps << ','
            << row.iters << ','
            << row.x << ','
            << row.absError << ','
            << row.residual << '\n';
    }
}

void WriteConditioningCsv(const std::string& filename,
                          const std::vector<ConditioningRow>& rows) {
    std::ofstream out(filename);
    out << "delta,status,iters,x,abs_error,residual\n";
    out << std::setprecision(17);
    for (const auto& row : rows) {
        out << row.delta << ','
            << (row.ok ? "ok" : "nmax") << ','
            << row.iters << ','
            << row.x << ','
            << row.absError << ','
            << row.residual << '\n';
    }
}

}  // локальное пространство имён

int main() {
    try {
        std::cout << std::fixed << std::setprecision(15);

        const double q = EstimateQ(dphi, LEFT, RIGHT);
        std::cout << "Function: f(x) = exp(1/x^2) - ln(x)\n";
        std::cout << "Interval: [" << LEFT << "; " << RIGHT << "]\n";
```

```
std::cout << "Initial approximation x0 = " << X0 << "\n";
std::cout << "lambda = " << LAMBDA << "\n";
std::cout << "q = max |phi'(x)| on [" << LEFT << "; " << RIGHT << "] = " << q << "\n\n";

const auto reference = ITER(phi, X0, 1e-14, NMAX);
const double xref = reference.x;
std::cout << "Reference run (delta = 0, eps = 1e-14):\n";
std::cout << "  xref  = " << xref << "\n";
std::cout << "  iters = " << reference.iters << "\n";
std::cout << "  |f(xref)| = " << std::fabs(f(xref)) << "\n\n";

const std::vector<double> epsValues = {
    1e-1, 1e-2, 1e-3, 1e-4,
    1e-5, 1e-6, 1e-7, 1e-8
};

std::vector<SpeedRow> speedRows;
std::cout << "Speed of convergence:\n";
for (double eps : epsValues) {
    const auto result = ITER(phi, X0, eps, NMAX);
    const SpeedRow row{
        eps,
        result.iters,
        result.x,
        std::fabs(result.x - xref),
        std::fabs(f(result.x))
    };
    speedRows.push_back(row);

    std::cout << "  eps = " << std::scientific << eps
              << ", iters = " << std::fixed << row.iters
              << ", x = " << std::setprecision(15) << row.x
              << ", |x - xref| = " << std::scientific << row.absError
              << ", |f(x)| = " << row.residual << '\n';
}
std::cout << '\n';

const std::vector<double> deltaValues = {
    1e-1, 1e-2, 1e-3, 1e-4,
    1e-5, 1e-6, 1e-7, 1e-8
};
```

```
        std::vector<ConditioningRow> conditioningRows;
        std::cout << "Conditioning experiment (eps = 1e-8):\n";
        for (double delta : deltaValues) {
            const auto result = ITER(
                [delta](double x) {
                    return phiRounded(x, delta);
                },
                X0,
                1e-8,
                NMAX
            );

            const ConditioningRow row{
                delta,
                result.iters,
                result.x,
                std::fabs(result.x - xref),
                std::fabs(f(result.x)),
                result.ok
            };
            conditioningRows.push_back(row);

            std::cout << "  delta = " << std::scientific << delta
                      << ", status = " << (row.ok ? "ok" : "nmax")
                      << ", iters = " << std::fixed << row.iters
                      << ", x = " << std::setprecision(15) << row.x
                      << ", |x - xref| = " << std::scientific << row.absError
                      << ", |f(x)| = " << row.residual << '\n';
        }

        WriteSpeedCsv("speed_results.csv", speedRows);
        WriteConditioningCsv("conditioning_results.csv", conditioningRows);

        return 0;
    } catch (const std::exception& ex) {
        std::cerr << "Error: " << ex.what() << '\n';
        return 1;
    }
}
```

Название файла: plot.py

```
import csv
```

```
import math
from pathlib import Path

import matplotlib.pyplot as plt


def f(x: float) -> float:
    return math.exp(1.0 / (x * x)) - math.log(x)


def read_csv(path: str):
    with open(path, newline='', encoding='utf-8') as fobj:
        return list(csv.DictReader(fobj))


base = Path('.')
speed_rows = read_csv(base / 'speed_results.csv')
cond_rows = read_csv(base / 'conditioning_results.csv')

# ------------------------------------------------------------
# Рисунок 1: график функции и отрезок отделения корня
# ------------------------------------------------------------
xs = [2.4 + i * (2.0 / 500.0) for i in range(501)]
ys = [f(x) for x in xs]

plt.figure(figsize=(7.2, 4.6))
plt.plot(xs, ys)
plt.axhline(0.0)
plt.axvline(3.0, linestyle='--')
plt.axvline(4.0, linestyle='--')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('f(x) = exp(1/x^2) - ln(x)')
plt.tight_layout()
plt.savefig('figure_function.png', dpi=220)
plt.close()

# ------------------------------------------------------------
# Рисунок 2: число итераций в зависимости от требуемой точности
# ------------------------------------------------------------
xs_speed = [-math.log10(float(row['eps'])) for row in speed_rows]
ys_speed = [int(row['iters']) for row in speed_rows]
```

```
plt.figure(figsize=(7.2, 4.6))
plt.plot(xs_speed, ys_speed, marker='o')
plt.xlabel('-log10(Eps)')
plt.ylabel('Число итераций')
plt.title('Метод простых итераций: скорость сходимости')
plt.tight_layout()
plt.savefig('figure_speed.png', dpi=220)
plt.close()

# ------------------------------------------------------------
# Рисунок 3: абсолютная погрешность в зависимости от шага округления
# ------------------------------------------------------------
xs_cond = [float(row['delta']) for row in cond_rows if row['status'] == 'ok']
ys_cond = [float(row['abs_error']) for row in cond_rows if row['status'] == 'ok']

plt.figure(figsize=(7.2, 4.6))
plt.loglog(xs_cond, ys_cond, marker='o')
plt.xlabel('Delta')
plt.ylabel('|x_delta - x_ref|')
plt.title('Чувствительность к округлению в phi(x)')
plt.tight_layout()
plt.savefig('figure_conditioning.png', dpi=220)
plt.close()
```